

Week 3 · Student Notes

AI531C Digital Image Analysis · Tue 18 – Wed 19 Aug 2026 · Unit I · Lectures 7, 8, 9

Everything taught in this week, in one document. The per-lecture handouts are identical in content — use whichever you prefer.

What is in this bundle

- Lecture 7 — Histogram Analysis, Equalisation and Adaptive Contrast Enhancement
- Lecture 8 — Two-Dimensional Correlation, Convolution and Spatial Filtering
- Lecture 9 — Fourier-Domain Image Analysis and Multi-Scale Representation

This week at a glance

Sessions — Tue 18 – Wed 19 Aug 2026.

Tutorial — T3 (Tue 18 Aug).

Pre-class preparation — PCM-3.

Released after the last class of the week: Wed 19 Aug 2026, 18:00.

How to use this bundle — read the lecture section on the evening of the class while the argument is fresh, attempt the check-your-understanding questions before the tutorial, and leave the practice questions until you can do them unaided. No answer keys are printed: the reasoning is worked through in the tutorial.

Lecture 7 · Histogram Analysis, Equalisation and Adaptive Contrast Enhancement

AI531C Digital Image Analysis · Unit I · CO1 · M.Tech. Artificial Intelligence, Department of CSE, DTU

Last week you chose the curve; today the image chooses its own — the histogram becomes the designer

What you should already know

This lecture continues from Lecture 6, “Point Operations and Intensity Transformations”. Revise that material first — today's arguments assume it.

Reading in the course notes: Comprehensive Notes L8; Vol. I ch. 8.

Learning outcomes

After studying this lecture you should be able to:

Histogram as probability, CDF as curve

Problem: build an intensity transform from the image itself. Treat the $M \times N$ image's intensities as samples of a random variable. L levels, $k = 0 \dots L-1$.

$$p(r_k) = n_k / (M \cdot N) \quad s_k = \text{round}[(L-1) \cdot \sum_{j \leq k} p(r_j)]$$

What each symbol means

- n_k — number of pixels with intensity r_k
- $p(r_k)$ — empirical probability (normalised histogram)
- $\sum p(r_j)$ — the CDF at level k , in $[0, 1]$
- s_k — output level assigned to every pixel at r_k

How to read this

What it means — The CDF rises fastest where pixels are densest — so the mapping is steepest exactly there. In L6 language: contrast gain is spent where the histogram mass is. That is the whole trick.

Worked example

Given — 64×64 image ($MN = 4096$), $L = 8$ levels, counts $n_k = [790, 1023, 850, 656, 329, 245, 122, 81]$ (GW4 Ch. 3 example).

Find — The equalisation mapping s_k for every level.

Formula — $p_k = n_k/4096$; $s_k = \text{round}(7 \cdot \text{CDF}_k)$

Solution

Working — $p_k = [.19, .25, .21, .16, .08, .06, .03, .02]$ CDF = $[.19, .44, .65, .81, .89, .95, .98, 1.00]$
 $7 \cdot \text{CDF} = [1.35, 3.10, 4.58, 5.68, 6.24, 6.66, 6.86, 7.00]$ ANSWER: $s_k = [1, 3, 5, 6, 6, 7, 7, 7]$

What it means — Dense dark levels 0-2 are pushed apart (0→1, 1→3, 2→5); sparse bright levels 4-7 merge (three levels share 7). Contrast is taken from where pixels are rare and given to where they are common.

The code behind the results

```
eq = cv2.equalizeHist(img)
# global clahe = cv2.createCLAHE( clipLimit=2.0, tileGridSize=(8, 8))
out = clahe.apply(img)
# local # colour images: apply on V/L only (L5!)
```

Parameters used — clipLimit 1-8; tileGridSize 8×8 . Full program: L07_histogram_equalisation.py

What it means — Clip limit caps each tile's histogram peak — capping the local slope, hence the noise gain.

Something to think about — clipLimit = 8? Flat sky turns grainy: amplified noise, the L6 failure returning locally.

Global HE vs AHE vs CLAHE

Method	Statistics used	Strength	Characteristic failure
Global HE	One histogram for the image	Fast; parameter-free; full range used	Uneven illumination: one region's fix is another's ruin
AHE	One histogram per tile	Local detail everywhere	Amplifies noise in flat tiles; block seams
CLAHE	Per-tile, clipped, blended	Local detail with bounded noise gain	Clip limit too high → graininess; too low → little effect

Key takeaway: CLAHE is AHE with the noise-amplification failure engineered out — the clip limit IS the noise budget.

Core concepts

Failure cases: what the histogram cannot see

The histogram is blind to position — Two utterly different images can share one histogram — a gradient and a shuffled gradient. Equalisation treats them identically; no histogram method can use spatial structure. (Spatial methods start tomorrow.)

Over-enhancement of good images — HE always stretches toward uniform — applied to a well-exposed image it exaggerates noise and shifts tones for no benefit. Enhancement needs a reason, not a habit.

Bimodal domination — A document image (many white, many black pixels) gives a CDF that spends nearly the whole output range on the two peaks — the mid-tone strokes you cared about get squeezed. Local or targeted methods win there.

Where this is used — your project

Low-light lane — HE and CLAHE (on V) join gamma/log as the graded classical baselines for L33 — report all of them in the same metric table;

Every lane — output-histogram inspection is your standing QA step — clipping spikes at 0/255 mean your pipeline destroyed information (L6's warning, now with a tool); dataset cards: include a pooled intensity histogram per split — distribution mismatch between train and test shows up here first (returns quantitatively in L25/L27).

Try this yourself — CLAHE clip-limit sweep {1, 2, 4, 8} on your darkest project image; record the value where noise first becomes objectionable.

Summary

- The histogram is the image's evidence file — read it before processing anything. Equalisation = let the CDF place the contrast gain where the pixels are. CLAHE bounds the local slope — enhancement with an explicit noise budget.

Key formulae

$p(r_k) = n_k / MN$ $s_k = \text{round}[(L-1) \cdot \sum_{j \leq k} p(r_j)]$ Continuous: $T(r) = (L-1) \cdot \int_0^r p(w) dw \rightarrow$ uniform CLAHE: clipLimit caps per-tile slope

What comes next

Tutorial T3 — today 2:00–3:00 PM (studio) — Spatial filtering and convolution studio: bring your PCM-3 kernel prediction card. You will implement conv2d, verify it against OpenCV to 0.0, and fill the parameter-effect table. The contrast section uses today's CLAHE on your own images.

Before tomorrow (L8–L9 double lecture) — Nothing new — review your prediction card after the studio; tomorrow's lecture opens with the mismatches.

Logistics — L8 (Wed 10 AM): correlation, convolution, spatial filtering. L9 (Wed 11 AM): Fourier-domain analysis and pyramids.

Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

1. Diagnosis: match histogram to image

From your PCM-3 histogram item and the panel on the earlier slide: 1. Which histogram shape says 'under-exposed', which says 'low contrast', which says 'clipped'? 2. Predict: what does the histogram of a well-equalised image look like — and why is it NOT flat in practice? 3. Sketch the histogram of an image that equalisation would make WORSE.

Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, understand): Derive, in the continuous setting, why the scaled CDF equalises a histogram, and state the discrete-case caveat.

Q2 (6 marks, apply): A 4-level image has $p = [0.6, 0.2, 0.1, 0.1]$. Compute the equalisation mapping s_k and comment on what happens to levels 0 and 1.

Q3 (10 marks, analyse): Night CCTV frames show a bright lamp region and large dark areas. Global HE makes the lamp bloom and the shadows noisy. Explain both artefacts and design a better enhancement, justifying every parameter.

Further reading

GW4 Ch.3 Sec.3.3; Szeliski Ch.3 Sec.3.1

Course notes: Comprehensive Notes L8; Vol. I ch. 8.

Runnable code for every figure and number in this lecture: 06_CODE_AND_DEMOS (see the file names quoted beside each figure).

AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.

Lecture 8 · Two-Dimensional Correlation, Convolution and Spatial Filtering

AI531C Digital Image Analysis · Unit I · CO1 · M.Tech. Artificial Intelligence, Department of CSE, DTU

The single most important operation in this course — you implemented it yesterday; today we understand it

What you should already know

This lecture continues from Lecture 7, “Histogram Analysis, Equalisation and Adaptive Contrast Enhancement”. Revise that material first — today's arguments assume it.

Reading in the course notes: Comprehensive Notes L9 (spatial part); Vol. I ch. 9.

Learning outcomes

After studying this lecture you should be able to:

The operation, precisely

Problem: compute an output pixel from a weighted neighbourhood. Kernel w of size $(2a+1) \times (2b+1)$; image f ; assume reflect padding at borders.

$$g(x, y) = \sum_s \sum_t w(s, t) \cdot f(x - s, y - t)$$

What each symbol means

- $w(s, t)$ — kernel weights, $s \in [-a, a]$, $t \in [-b, b]$ f, g — input and output images Minus signs — the kernel FLIP that distinguishes convolution Correlation drops the flip: $f(x + s, y + t)$

How to read this

What it means — Linear (superposition holds) and shift-invariant (same rule everywhere) — the two properties the frequency view of L9 depends on. For symmetric kernels flip changes nothing — why `cv2.filter2D` (correlation) 'worked' in T3.

Worked example

Given — 4×4 patch $f = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 3 & 1 \\ 2 & 2 & 1 & 0 \\ 1 & 0 & 0 & 2 \end{bmatrix}$; sharpen kernel $w = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$; zero padding outside.

Find — Outputs at the four interior positions (1,1), (1,2), (2,1), (2,2).

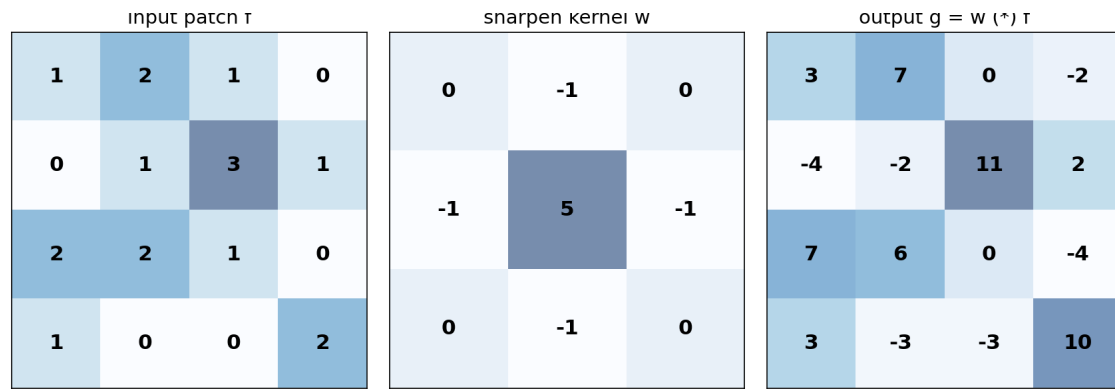
Formula — $g = 5 \cdot \text{centre} - (\text{up} + \text{down} + \text{left} + \text{right})$ (w is symmetric: flip irrelevant)

Solution

Working — $g(1,1) = 5 \cdot 1 - (2+2+0+3) = -2$ $g(1,2) = 5 \cdot 3 - (1+1+1+1) = 11$ $g(2,1) = 5 \cdot 2 - (1+0+2+1) = 6$ $g(2,2) = 5 \cdot 1 - (3+0+2+0) = 0$ ANSWER: $g_{\text{interior}} = \begin{bmatrix} -2 & 11 \\ 6 & 0 \end{bmatrix}$

What it means — The bright pixel (3) is pushed up to 11; its dimmer neighbour is pushed negative — sharpening manufactures overshoot on both sides of an edge. Negative outputs force a range decision: clip, rescale, or keep float.

Figure 1: The mechanics, frame by frame



Input patch, sharpen kernel, full output (zero padding). Generated by L08_spatial_filtering.py.

How to read this figure

What to notice — Interior outputs match the hand calculation; border outputs differ.

Why it happens — At borders the kernel hangs off the image — padding invents the missing neighbours.

What it means — Border values are part data, part convention — every filtered image has a part-fiction frame one kernel-radius wide.

Why it matters — Never measure anything within a kernel radius of the border.

The code behind the results

```
blur = cv2.blur(img, (5, 5))
gauss = cv2.GaussianBlur(img, (5,5), 1.2)
sharp = cv2.filter2D(img, -1, K_sharp)
med = cv2.medianBlur(sp_noisy, 5)
bilat = cv2.bilateralFilter(img, 9, 60, 10)
# ddepth=-1 keeps uint8: negatives CLIP
```

Parameters used — size 5×5; $\sigma=1.2$; bilateral d=9, $\sigma_{\text{colour}}=60$, $\sigma_{\text{space}}=10$. Full program: L08_spatial_filtering.py

What it means — Six lines cover the classical smoothing/sharpening toolbox.

Something to think about — ddepth=cv2.CV_32F? Negatives survive — mandatory for gradients (L15).

Filter selection by evidence

Filter	Linear?	Right when	Wrong when
Box	Yes (separable)	Speed matters, mild noise	Edges matter — smears them worst
Gaussian	Yes (separable)	Gaussian-like noise; pre-filter for sampling (L3)	Impulse noise — averages the spikes in
Median	No	Salt & pepper; thin-edge preservation	Gaussian noise (weaker); fine texture (eats it)
Bilateral	No	Denoise while keeping edges	Heavy noise (kernel misled); real-time budgets
Sharpen	Yes	Mild defocus, display crispness	Any noisy input — amplifies it (L6 slope logic)

Key takeaway: the noise model chooses the filter — 'which filter is best' is an ill-posed question until the degradation is named (L10 formalises this).

Where this is used — your project

Denoising lane (primary) — Gaussian/median/bilateral ARE your classical baselines — the T5 studio benchmarks them with PSNR/SSIM, and Unit IV's CNN denoiser is graded against them;

All lanes — any resize, any gradient, any 'feature' you compute starts life as a kernel — log the kernel sizes and σ values in your dataset card; border policy: decide it once (reflect-101 recommended), write it down, use it everywhere — mixed borders create phantom differences between 'identical' pipelines.

Try this yourself — Bilateral σ_{colour} sweep {20, 60, 150} on one project image; find where it starts erasing real structure.

Summary

- Convolution = flipped, shifted, weighted sums — linear and shift-invariant. Separability turns k^2 work into $2k$ — check the kernel's rank before optimising anything else. Borders are convention, not data; noise type — not taste — picks the filter.

Key formulae

$$g(x,y) = \sum_s \sum_t w(s,t) \cdot f(x-s, y-t) \text{ Separable: } w(s,t) = w_1(s) \cdot w_2(t); \text{ cost } k^2 \rightarrow 2k \text{ Unsharp: } g = f + \lambda(f - f_{\text{blur}})$$

What comes next

Immediately next (L9, same room) — Fourier-domain analysis: the frequency identity of every kernel you used today, plus notch filtering and pyramids.

PCM-4 (releases Fri 21 Aug, due Mon 24 Aug) — Spectrum and periodic-noise puzzle + the ten-lecture concept checklist — it doubles as Assessment Test I preparation (test at T4, Tue 25 Aug, after L10).

Logistics — Tutorial T4 (Tue 25 Aug): Assessment Test I + Fourier studio. Question Bank 1 topics (L1–L3) are in scope for the test; L4–L10 join it.

Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

1. Prediction card debrief

Take out your PCM-3 card and your T3 outputs: 1. Where did your averaging-kernel prediction disagree with the actual output? 2. Most common miss in past runs: what happens at the EDGE under K1 — did you predict blur or did you predict a new grey band? 3. Explain the miss with one phrase from today: flip, superposition, overshoot, or padding.

Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, understand): State the difference between correlation and convolution, and name a kernel for which it matters and one for which it does not.

Q2 (6 marks, apply): Show that a 31×31 box kernel is separable and compute the multiplication counts per pixel, direct vs separated.

Q3 (10 marks, analyse): A student sharpens a photo, saves it as uint8, then computes gradients for edge detection and reports 'half the edges vanished'. Explain the mechanism precisely and give the two-word fix.

Further reading

GW4 Ch.3 Secs.3.4–3.7; Szeliski Ch.3 Secs.3.2–3.3; Forsyth-Ponce filtering

Course notes: Comprehensive Notes L9 (spatial part); Vol. I ch. 9.

Runnable code for every figure and number in this lecture: 06_CODE_AND_DEMOS (see the file names quoted beside each figure).

AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.

Lecture 9 · Fourier-Domain Image Analysis and Multi-Scale Representation

AI531C Digital Image Analysis · Unit I · CO1 · M.Tech. Artificial Intelligence, Department of CSE, DTU

Every kernel has a second identity; every image has a spectrum — and some problems that look hopeless in space are trivial in frequency

What you should already know

This lecture continues from Lecture 8, “Two-Dimensional Correlation, Convolution and Spatial Filtering”. Revise that material first — today's arguments assume it.

Reading in the course notes: Comprehensive Notes L9 (frequency part); Vol. I ch. 9.

Learning outcomes

After studying this lecture you should be able to:

The 2-D DFT and the convolution theorem

Problem: represent an $M \times N$ image as a weighted sum of 2-D sinusoids. Frequencies $u \in [0, M-1]$, $v \in [0, N-1]$; spectra shown centred (fftshift).

$$F(u,v) = \sum_x \sum_y f(x,y) \cdot e^{-j2\pi(ux/M + vy/N)} \quad g = w * f \Leftrightarrow G = W \cdot F$$

What each symbol means

- $F(u, v)$ — complex coefficient of the (u, v) grating $|F|$ — magnitude (how much of that frequency); $\angle F$ — phase (where it sits) W — the kernel's spectrum: its frequency response $*$ — convolution; $\cdot$ — pointwise product

How to read this

What it means — Convolution in space = multiplication in frequency: every L8 kernel is secretly a mask on the spectrum. Phase carries WHERE structure sits — discard it and the image is unrecognisable; magnitude alone is nearly enough only for textures.

Worked example

Given — A 256×256 image is corrupted by horizontal stripes repeating every 8 pixels vertically (as in the panel).

Find — Where the interference appears in the centred spectrum, and where to place the notch filters.

Formula — Frequency index = N / period (cycles per image), on the axis of variation

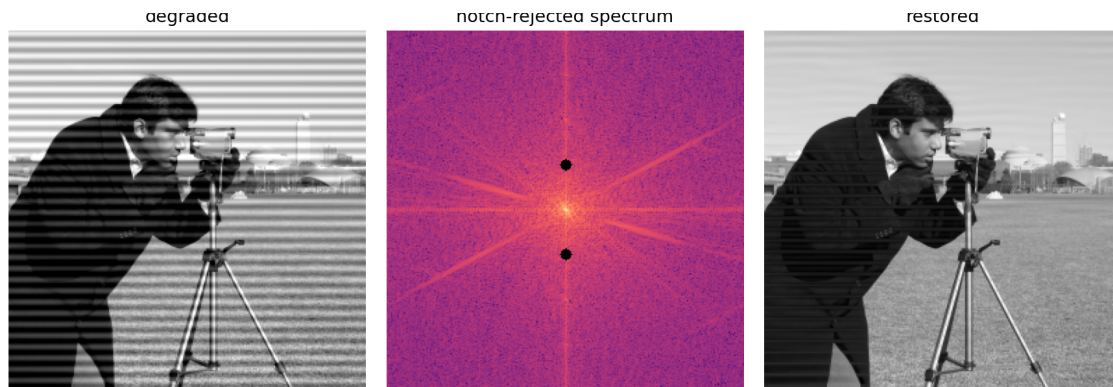
Solution

Substitution — Index = $256 / 8 = 32$; variation is vertical $\rightarrow$ spikes on the vertical frequency axis

Calculation — Centred coordinates: $(u, v) = (0, +32)$ and $(0, -32)$ — conjugate pair ANSWER: two notches at $v = \pm 32$ on the vertical axis, radius ≈ 4

What it means — The pair is symmetric because the image is real-valued. Small notches surgically delete the stripes while touching almost nothing else — verified in the next figure.

Figure 1: The notch at work



Degraded input, notch-rejected spectrum, restored output. Generated by L09_frequency_filtering.py.

How to read this figure

What to notice — Two tiny black disks in the spectrum; stripes gone in the output; scene detail intact.

Why it happens — The interference occupied exactly those coefficients; the scene barely used them.

What it means — Filtering is masking in frequency — this is $G = H \cdot F$ made visible.

Why it matters — Any periodic degradation (sensor hum, screen moiré, halftone) dies the same way.

The code behind the results

```
Fs = np.fft.fftshift(np.fft.fft2(img))
mag = np.log1p(np.abs(Fs))
# L06 log! H = np.ones((N, N)) H[dist_from((0, +32)) <= 4] = 0 H[dist_from((0, -32)) <= 4] = 0
out = np.real(np.fft.ifft2( np.fft.ifftshift(Fs * H)))
```

Parameters used — notch centres ± 32 ; radius 4. Full program: L09_frequency_filtering.py

What it means — Transform $\rightarrow$ mask $\rightarrow$ inverse: the entire frequency-domain workflow in nine lines.

Something to think about — Radius 40 instead of 4? Real image content dies too — over-notching blurs.

Spatial vs frequency filtering — when to go where

Question	Stay spatial (L8)	Go frequency (L9)
Kernel size	Small (3–15): direct conv is fastest	Large ($\geq \sim 31$): FFT route wins ($\sim 36\times$ at $k=31, N=512$)
Degradation type	Local noise, local edits	Periodic/global patterns — they concentrate into spikes
Design intuition	Weights over neighbours	Pass/reject bands and notches
Non-linear ops	Median, bilateral live here only	No frequency identity — theorem needs linearity

Key takeaway: same operations, two coordinate systems — choose the one where your problem is sparse.

Where this is used — your project

Denoising lane — if your degradation has ANY periodic component (sensor banding, flicker), notch it FIRST — no CNN should waste capacity learning what two notches remove;

Low-light lane — homomorphic filtering is a legitimate classical baseline next to gamma/CLAHE — three lectures of tools now compose;

Fusion / super-resolution lanes — pyramids are your workhorse (L40 fusion rule; SR's scale-space reasoning) — implement pyrDown/pyrUp fluently now.

Try this yourself — FFT your project images; look for structure you did not know was there (JPEG blocking shows up as a grid of spikes).

Summary

Today — Convolution $\Leftrightarrow$ multiplication; phase holds position; periodic noise dies by notch; log turns $f = i \cdot r$ into a sum; pyramids = recursive safe downsampling.

Unit I in one breath — Images are sampled, quantised measurements (L1–L3), structured by adjacency (L4), represented in chosen spaces (L5), remapped by curves (L6–L7), and filtered by neighbourhoods in space or frequency (L8–L9).

Key formulae

$g = w * f \Leftrightarrow G = W \cdot F$ Notch index = N / period , conjugate pair $\ln f = \ln i + \ln r \rightarrow \text{high-pass} \rightarrow \exp$ Pyramid: blur ($\sigma \approx 1$)
 $\rightarrow \downarrow 2 \rightarrow \text{repeat}$

What comes next

PCM-4 (Fri 21 Aug $\rightarrow$ Mon 24 Aug, ~ 30 min) — Spectrum and periodic-noise puzzle + ten-lecture concept checklist — direct test preparation.

Tuesday 25 Aug — L10 (12 PM): Unit II opens — image degradation model and noise processes. T4 (2–3 PM): Assessment Test I (L1–L10 concepts, numericals, interpretation) followed by the Fourier studio.

Logistics — Test: 5 CWS marks; conceptual 2.0 + numerical 2.0 + interpretation 1.0. Revision: Question Bank 1 + each deck's summary and exam slides.

Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

1. Interpret this spectrum

A photograph of a laptop screen shows faint diagonal bands. Predict: 1. Where do the corresponding spikes sit in the centred spectrum (axis? off-axis?) 2. The bands repeat every ~ 6 px in a 512-px image — at what radius from centre? 3. What happens if you notch only ONE of the two spikes?

Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, understand): State the convolution theorem and explain why the median filter has no frequency-domain equivalent.

Q2 (6 marks, apply): A 512×512 scan shows vertical stripes repeating every 16 pixels horizontally. Give the centred spectrum coordinates of the interference and design the notch mask.

Q3 (10 marks, analyse): A microscopy facility 'fixes' uneven illumination by dividing each image by a heavily blurred copy of itself. Relate this to homomorphic filtering, analyse when it works, when it fails, and which is preferable.

Further reading

GW4 Ch.4; Szeliski Ch.3 Secs.3.4–3.5

Course notes: Comprehensive Notes L9 (frequency part); Vol. I ch. 9.

Runnable code for every figure and number in this lecture: 06_CODE_AND_DEMOS (see the file names quoted beside each figure).

AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.